МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра Информационных Систем

ОТЧЕТ
по лабораторной работе №1
по дисциплине «ООП»
Тема: Введение в ООП. Создание ММО РПГ с нуля

Студент гр. 5372 _____ Ермаков Д. В.

Преподаватель _____ Глущенко А.Г.

Санкт-Петербург
2026

Цель работы

Освоить основы ООП (классы, наследование, контейнеры) на примере разработки игры и различных юнитов. Реализовать паттерны **«Фабричный метод»** / **«Абстрактная фабрика»** для создания юнитов, а также **итератор** для обхода юнитов на игровом поле.

Язык реализации - C++

Основные требования к программе:

1. Игровое поле (GameField). Создание поля произвольного размера, хранение юнитов, ограничение их количества, добавление и удаление объектов, получение доступа к ним.
2. Иерархия юнитов (Unit). Создание абстрактного базового класса и нескольких типов конкретных юнитов с общим интерфейсом.
3. Система характеристик. Реализация отдельных классов Health, Armor и Attack для управления здоровьем, бронёй и силой атаки.
4. Создание объектов. Использование порождающего паттерна для создания юнитов через фабрики.
5. Обход объектов. Реализация собственного итератора для последовательного обращения к юнитам игрового поля.
6. Управление памятью. Реализация глубокого копирования, операций перемещения и использование умных указателей.

Архитектура программы:

Программа построена на основе объектно-ориентированного подхода. Основой системы является абстрактный класс Unit, от которого наследуются классы различных категорий и конкретных типов юнитов. В проекте выделены три основные категории:

Assault - штурмовые юниты;

Defense - защитные юниты;

Support - юниты поддержки.

Каждая категория содержит два конкретных типа. Например, от класса Assault наследуются Soldier и Pyro, от Defense - Heavy и Engineer, от Support - Medic и Sniper.

Для хранения характеристик юнитов используются отдельные классы Health, Armor и Attack. Такое разделение позволяет независимо управлять здоровьем, защитой и наносимым уроном.

В программе используется объектно-ориентированный подход, при котором игровые объекты и их характеристики разделены на отдельные классы.

Инкапсуляция: Инкапсуляция применяется для ограничения прямого доступа к внутренним данным объектов. Например, классы Health, Armor и Attack хранят свои характеристики в закрытых полях private. Изменение значений осуществляется через специальные методы: increase(), decrease(), isZero().

Наследование: В программе реализована иерархия классов, основанная на общем базовом классе Unit. От него наследуются три промежуточных класса: Assault, Defense и Support. Каждый из них объединяет конкретные виды юнитов соответствующей категории.

Полиморфизм: Полиморфизм реализован с помощью виртуальных методов базового класса Unit. Каждый конкретный юнит самостоятельно реализует методы getType() и clone(). При этом игровое поле хранит объекты через указатели на базовый класс.

Абстракция: Абстракция представлена классами Unit и UnitFactory, которые определяют общие интерфейсы для своих наследников. Например, в Unit объявлены чисто виртуальные методы:

Реализация паттернов проектирования

В проекте реализованы три паттерна:

Фабричный метод (Factory Method) - создание юнитов вынесено в отдельные фабрики AssaultFactory, DefenseFactory и SupportFactory, наследуемые от UnitFactory. Метод createUnit() возвращает объект через базовый класс Unit, скрывая конкретный тип создаваемого юнита.

Прототип (Prototype) - в классе Unit предусмотрен виртуальный метод clone(), переопределяемый в производных классах. Он позволяет создавать копии юнитов с сохранением их типа и характеристик.

Итератор (Iterator) - класс GameFieldIterator обеспечивает последовательный обход юнитов, предоставляя методы получения следующего элемента и проверки его наличия.

Таким образом, паттерны отвечают за создание, копирование и обход объектов.

Почему был выбран Фабричный метод?
В работе я придерживался фабричного метода. всего у меня три категории юнитов, которые выполняют свою роль в бою. например, защитой занимаются пулемётчик (Heavy) и инженер (Engineer), а поддержкой занимаются медик (Medic) и снайпер (Sniper).

Для каждой категории своя фабрика: AssaultFactory, DefenseFactory и SupportFactory. все они наследуются от общего абстрактного UnitFactory, который определяет общий интерфейс создания юнитов. таким образом, логика создания объектов разделяется, GameField принимает любую фабрику не обращаясь напрямую к конкретному классу.

Также при добавлении новых типов юнитов можно расширить систему фабрик, не меняя логику игрового поля

Игровое поле и управление памятью

Класс GameField отвечает за хранение и управление юнитами. Для этого используется контейнер: **std::vector<std::unique_ptr<Unit>> units;**

Умные указатели std::unique_ptr обеспечивают автоматическое освобождение памяти. Игровое поле поддерживает добавление, удаление и получение юнитов, а также проверку ограничений.

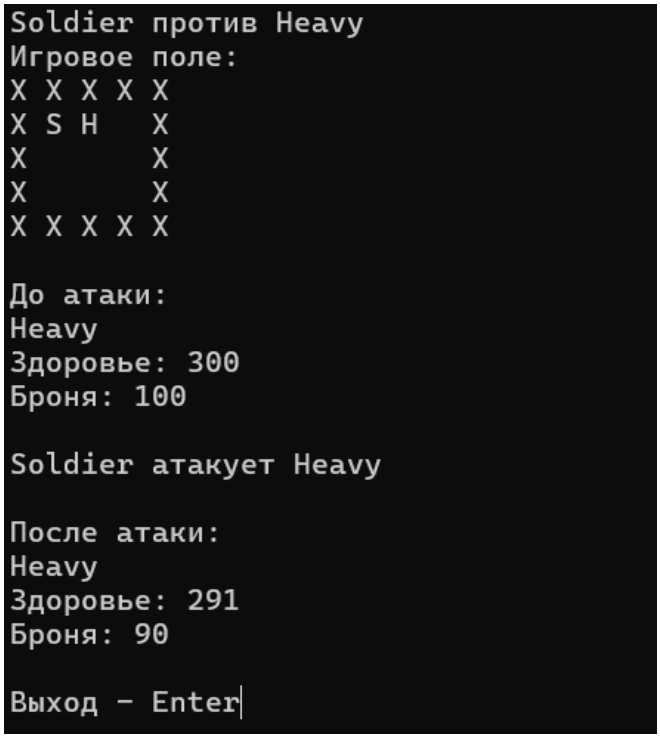
Для копирования объектов используется метод clone(), позволяющий создавать независимые копии юнитов. При перемещении поля его ресурсы передаются новому объекту без дополнительного копирования.

В GameField реализовано "Правило пяти": конструктор и оператор копирования, конструктор и оператор перемещения, а также деструктор. Это обеспечивает корректное управление памятью при работе с игровым полем.

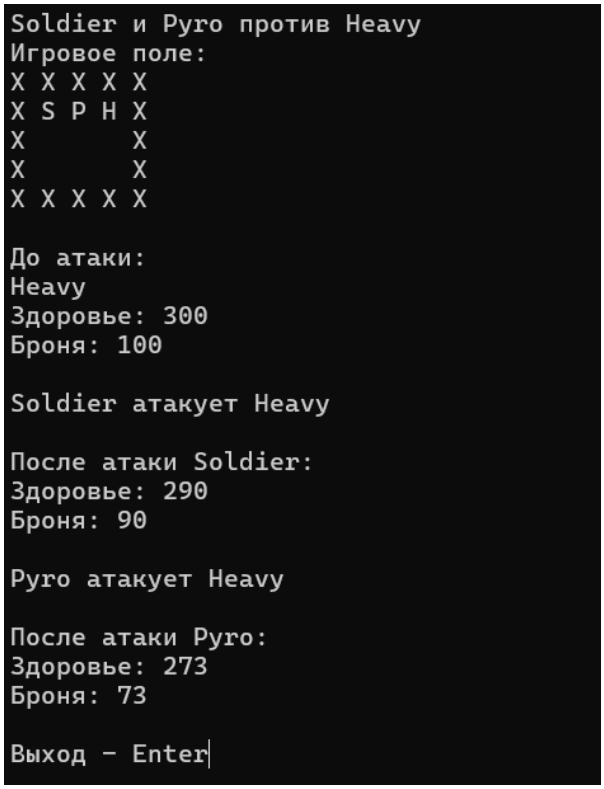
Демонстрационные сценарии программы

В функции main() представлены несколько сценариев, демонстрирующих основные возможности программы.

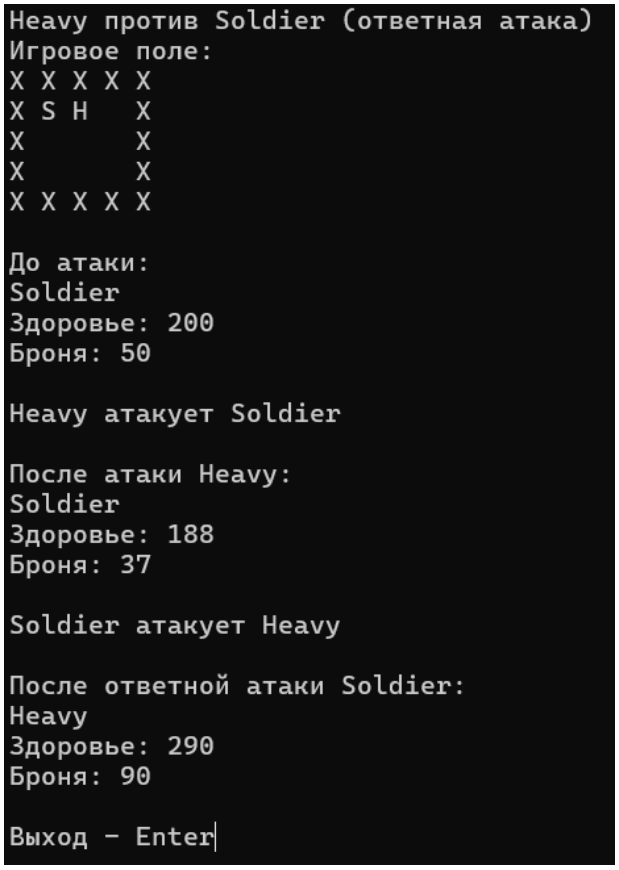
Сначала создаются юниты разных типов, например Soldier и Heavy, после чего проверяется их взаимодействие в бою. Один юнит наносит удар другому, при этом учитываются характеристики атаки, здоровья и брони.



В следующем сценарии демонстрируется атака нескольких юнитов и ответные действия противника. Это позволяет проверить работу боевых методов и изменение характеристик объектов в процессе взаимодействия.



В заключительной части демонстрируется перемещение юнитов по игровому полю и последующее взаимодействие между ними. Таким образом, программа показывает создание объектов, работу боевой системы и изменение положения юнитов.



Заключение

В ходе лабораторной работы была разработана игровая система на C++ с применением принципов ООП. Реализованы иерархия классов, паттерны проектирования, игровое поле и механизмы управления памятью.

В результате работы получены практические навыки использования наследования, полиморфизма, умных указателей и динамической памяти.